

Prague University of Economics and Business

Faculty of Informatics and Statistics



AUTOMATED MOBILE GAME TESTING

BACHELOR THESIS

Study programme: Applied Informatics

Field of study: Applied Informatics

Author: Daria Beliavskaia

Supervisor: Ing. et Ing. Michal Doležel, Ph.D.

Prague, December 2021

Declaration

I hereby declare that I am the sole author of the thesis entitled “Automated Mobile Game Testing.” I duly marked out all quotations. The used literature and sources are stated in the attached list of references.

In Prague on

Signature

Daria Beliavskaia

Acknowledgment

First of all, I would like to thank my supervisor *Ing. et Ing. Michal Doležel* for his guidance and patience. Without his help, that thesis would never exist. Then I want to thank me for believing in me and doing all this hard work. I want to thank me for having nonsleep nights and trying to do my best.

Abstract

The game industry has grown significantly in the last decade, particularly the mobile gaming segment, which now accounts for more than half of all applications available in the app stores. It is no surprise that there is a huge demand for higher-quality mobile game application.

Manual testing is still a common practise among mobile game developers, however today there is a range of test automation tools on the market, that can minimize the testing time and improve game quality and production time.

The main goal of this bachelor thesis is to compare test automation tools for mobile games by evaluating and implementing them into the game prototype. Each tool was evaluated according to the defined sets of criteria including test script development. Based on the results, the most relevant testing tool was selected.

Keywords

Mobile games, game testing, test automation, object recognition

Content

Introduction	10
Background.....	10
Aim and objectives	10
Overview and structure.....	11
Motivation.....	11
1 Mobile game testing	12
1.1 Importance of game testing.....	12
1.2 Differences between game testing and regular app testing	12
1.3 Challenges in mobile game testing	13
1.4 Methods for testing mobile games	14
1.4.1 Manual game testing.....	14
1.4.2 Automated game testing	14
2 Automated mobile game testing	15
2.1 The concepts of test automation for mobile games.....	15
2.1.1 White-box testing	15
2.1.2 Gray-box testing	16
2.1.3 Black-box testing	16
2.2 Limitations of mobile game test automation	17
3 Game sample prototype development.....	19
3.1 Game design.....	19
3.2 Test cases	20
4 Tools overview and definition	23
4.1 Selecting of test automation tools	23
5 Automation test tools evaluation	26
5.1 Defining evaluation criteria	26
5.2 Assign criteria weights using a ranking method	32
5.3 AltUnity Tester	33
5.3.1 AltUnity Tester evaluation	33
5.3.2 Implementation and testing.....	34
5.4 AirTestIDE.....	36
5.4.1 AirTestIDE evaluation.....	36
5.4.2 Implementation and testing.....	37
5.5 GameDriver	40

5.5.1 GameDriver evaluation	40
5.5.2 Implementation and testing	41
5.6 Results of the evaluation	43
Conclusions	45
List of references	46
Annexes.....	I
Annex A: Sample game's source code.....	I
Annex B: AltUnity Tester test script	I
Annex C: AirTestIDE test script	I
Annex D: GameDriver test script	I

List of Figures

Figure 1: Representative of the platform game [22]..... 20

Figure 2: Game controls21

Figure 3: Game enemies’ position..... 22

Figure 4: AirTestIDE coordinates system38

List of tables

Table 1: Set of the tool related criteria [15] [28]	27
Table 2: Set of the user-friendly related criteria	29
Table 3: Set of the test script related criteria	31
Table 4: Calculation of the weights of individual criteria.	32
Table 5: Evaluation of AltUnity Tester.	34
Table 6: AltUnity Tester test cases automation.	35
Table 7: Evaluation of AirTestIDE.	37
Table 8: AirTestIDE test cases automation.	39
Table 9: Evaluation of GameDriver.	41
Table 10: GameDriver test cases automation.	42
Table 11: Results of the evaluation.	43

List of program's codes

Code 1: AltUnity Tester script (access game objects)	35
Code 2: AltUnity Tester script (game mechanics test case)	36
Code 3: AltUnity Tester script (enemy test case)	36
Code 4: AirTestIDE script (access game objects)	39
Code 5: AirTestIDE script (attack test case)	39
Code 6: AirTestIDE script (mechanics test case)	40
Code 7: GameDriver script (access game objects)	42
Code 8: GameDriver script (mechanics test case)	43
Code 9: GameDriver script (enemy test case)	43

Introduction

Background

The game industry is in huge demand these days, and according to Business Wire [1], the global games sector will increase at a CAGR (Compound annual growth rate) of 12 % between 2020-2025. In 2020, the gaming industry generated \$159.3 billion, and half of the revenue came from the mobile segment. And it will not stop increasing, as during previous years, it was growing all the time. According to reports, the revenue has grown by up to \$39.2 billion since last year [2]. As can be seen, the games market is expanding rapidly, particularly in the mobile category, which accounts for more than half of the entire games market. Today, gaming apps are the most popular apps on both the Play Store and the App Store.

With the increasing demand for this kind of entertainment, game studios have started looking for more efficient and rapid ways of developing new products. Because testing is an essential aspect of game development and any type of software, companies have begun to carry out more tests that have become crucial, as the occurrence of bugs increases with the amount of code used.

The gaming industry was not ignored by test automation. However, for some reason, in such a wide area as game development, the majority of the tests are performed manually by the QA department. Game companies spend a lot of resources and time during the development and testing of their games, as they have a higher level of complexity than other software. However, in the recent few years, a few testing frameworks were created, that are specialized in game testing and are targeted at addressing the gap in test automation within the gaming industry. However, there has only been a few theses that mentioned those frameworks since then, and no comprehensive and systematic study or thesis on how other new tools can be used in game test automation. That work will fill the gap and describe the test automation possibilities in game development.

Aim and objectives

The main aim of this bachelor's thesis is to define and compare tools for automated mobile game testing and evaluate each tool.

The objectives of this thesis are:

1. Describe the concepts of game testing and approach the issue of automated testing.
2. Specify game mechanics and create a prototype mobile game for the purpose of evaluation.
3. Define the game test suites according to the game's mechanics.
4. Select appropriate tools for automated testing of mobile games.
5. Define the criteria to evaluate and compare the selected testing tools.

6. Develop test scripts, that cover test cases by using every selected tool and performing their execution.
7. Evaluate every tool with multicriteria analysis.

Overview and structure

The first part of the thesis deals with the testing of mobile games in general. The theoretical part is covered by Chapter 1 and Chapter 2. It describes the fundamentals and specifics of mobile game testing, as well as features that must be considered before testing, and challenges that developers and QA may face. In addition, that part covers automated testing and how it differs from manual testing, as well as how it can be used in game development and its limitations.

The technical part describes the creation of the game prototype, which will be tested in the following chapter. Its core functionalities and test suites are described in Chapter 3. The following chapter focuses on selecting the relevant testing tools and setting the acceptable criteria for mobile game testing. It also covers the thesis's final objectives, including the testing of the game prototype using the selected testing tools, as well as its evaluation and comparison.

Motivation

My thesis topic was inspired by my interest in game development. After getting familiar with automated mobile app testing, I decided to find out whether it is spreading among game companies. After spending some time on certain forums and reading a variety of blogs and research on that subject, I've come to the conclusion that automating testing in game development is not that common. Following that, I contacted a few Czech game companies, that develop mobile games and asked about automation. Even though some of their games were simple to automate, I was surprised to find no test automation in their projects. That is why I decided to write a thesis about mobile game testing automation in order to help mobile game developers to get familiar with automated testing and choose the best automation testing tool for their needs.

1 Mobile game testing

This chapter covers the relevance of mobile game testing as part of the game development life cycle. Then it goes on to highlight several game aspects that impact the testing process and distinguish games from mobile applications in general. These factors cause challenges that the QA department faces during the testing process, which are also described in the chapter. Following that, different types of mobile game testing are described, including automated testing, which will be covered further.

1.1 Importance of game testing

According to Charles P. Shultz, Robert Bryant, and Tim Langdell (2005), “Testing is important because of the contractual commitments and complex nature of the software required to deliver a topnotch game.” [3]

The majority of people who are not interested in game development believe that game testing is nothing more than simply playing. However, that is a time-consuming process, that requires a lot of effort to ensure that the final product meets all the requirements and provides a good user experience.

The main challenge of testing is to break the game in order to catch all of the code that is not working correctly. To accomplish that, the QA department needs to go through the game and ingeniously check its stability, since millions of players’ actions are possible. [4] To verify that the game works correctly, testers examine every possible interaction with objects, characters, environment, and other game elements, that might in a variety of sequences and combinations. That is why testing is critical in game development, especially at the early prototype stage, as it guarantees that all potential occurrences are covered.

1.2 Differences between game testing and regular app testing

All of the above raises the question of whether there are distinctions between testing a mobile game and testing a regular application. Although they both run on the same platforms as Android and IOS and testers have the same intention of testing software for bugs and defects, that the user may encounter, games have unique features, that impact tests strategies and make it rather challenging. It can be stated that game testing inherits nearly every aspect of software testing in general. There are a few features, though, that make game testing more difficult.

First, it is important to specify how the games differ from a technical perspective. Game development is the harder process since games have mechanics: the rules and procedures that guide the player, as well as the game’s response to the player’s moves or actions. In most of the cases, an application functions based on a set of algorithms. At least half of the mobile games do not have this, since it would entirely ruin the whole concept. That is why

games are more unpredictable than regular mobile applications since they allow users to act unpredictably, and the outcome is dependent on their actions. [5] Besides, the game app architecture might be complex, as programmers must build complex algorithms in order to implement the game designers' ideas. Consequently, complex errors may arise, making game testing more challenging. The complexity of the games is also the reason, why certain test cases are skipped, as it is both expensive and hard to test every potential combination. [6]

1.3 Challenges in mobile game testing

Since game testing is so complex, there are some challenges that QA teams may face. Mobile game testing, just like regular app mobile testing, has certain common difficulties that are similar in some ways to testing in both industries. The remaining challenges relate to game algorithms, unique user interfaces (UI), and interactivity:

1. Fragmentation is a problem that both games and applications face. Games must run on a wide variety of devices, each with its own set of operating systems, resolutions, dimensions, and screen settings. The game's performance may vary depending on the hardware and software used. It's also crucial to make sure that the reaction time and frame per rate (FPS) on any given piece of hardware satisfy the standards for a better user experience. [7]
2. Any game project is usually large and flexible, which means its lifespan differs from the usual application life cycle and changes on a frequent basis. As a result, games must be tested often.
3. Because of the frequent changes, there is a high chance that the new functionality will break the old one or that this functionality will disappear. It is difficult to detect it since it might appear anywhere in the code. [8]
4. A lack of time to test all states and combinations — as usual, testers do not use step-by-step cases since describing each action in the game takes a long time and leaves no time for actual testing. Instead, some game studios use checklists that cover all aspects of functionality. The most tested sets of test cases that are used in game development are the most appealing scenarios, which players are most likely to pass through.
5. The need to constantly monitor the size of the application, as the size limits are 150 MB for iOS and 100 MB for Android. The reason for this is that if the build exceeds that limit, iOS users will be unable to download the app using mobile data and would have to use a Wi-Fi network to do so. Android users will just receive an alert, but there is a chance to lose potential players. [9]
6. Difficulty of automation: there are very few games that are native, but most of the companies use game engines that are not supported by regular testing frameworks. Instead of displaying the game objects in the inspector, there is only one canvas, that does not provide access to and control over any game elements.
7. Testers' eyes glaze over—the most common problem for game testers since they must repeatedly play and test the same test cases. So that they stop detecting bugs or errors, that new users may notice.

1.4 Methods for testing mobile games

Mobile game testing may be either manual, in which case no test script coding is required, or automated, in which case scripts based on test cases run automatically and systematically report results.

1.4.1 Manual game testing

Manual testing is the most common method of testing used by gaming developers. That is the first thing that occurs during any game testing. To ensure that everything is checked, game testers go through the checklists, which include test cases covering various aspects and scenarios of the game.

The reason most companies prefer it is the lack of resources and skilled programmers to implement automated tests. However, some game features cannot be tested other than manually. Testing of the game balance and fun factor needs a special approach. To cover those aspects, play-testing is carried out. Its' goal is to test emotions and balance, determine how playable the game is and how much time the user spends in it, as well as identify those bugs or defects that the testers did not notice. These are the factors that affect the game's success after the actual release, therefore it is better to check them beforehand.

Although game testing is time-consuming and demanding due to the wide variety of possible game scenarios, it is good practice to use a manual approach for small game studios or individuals, whose games do not require a lot of testing and maintenance.

1.4.2 Automated game testing

Automated testing is defined as testing that is usually carried out through an automation tool and test scripts for the test scenarios. [10] It covers a few challenges that manual testing might cause. First, it solves the problem of device fragmentation and testing the games on different resolutions. The developed scripts can be run on many devices at once using special device farms. Then it saves time and eliminates the tiresome testing of repetitive test cases by creating reusable test scripts, that can be executed at any time. Instead, the QA department can focus on testing new features or more serious issues. Besides, growing functionality and frequent updates do not always allow manual testers to cover all regression cases in a short period of time. Moreover, the QA department may not be able to detect all issues, that may occur after the regression. The reason for this is that all game components are tightly interrelated and any changes to one might result in an error in another. That is why some game studios carry out automated testing, which allows them to run tests on a regular basis and detect bugs and errors before they are released, or even before they enter build. [11]

Nevertheless, the best practice in game testing is to combine manual testing and automated testing. Because every test scenario must be tested from several viewpoint at times, and it is not always possible to do so using single testing approach.

2 Automated mobile game testing

The chapter provides detailed information about automated testing and the options for its implementation into the game projects. It also covers the potential challenges in automated testing in game industry.

Before getting into the specifics of automated testing in the mobile game industry, it is important to gain an understanding of what automated testing is. Test automation is part of the testing process using software tools to execute tests and validate results. The goal of test automation is to speed up the testing process while maintaining the quality and allowing testers to focus on other high-priority tasks. [8] Before implementing the test automation into the game, it is important to distinguish between different types of mobile game test automation tools. Every game needs a different approach, so the developers and QA department must decide how they want to implement the test automation. Most of the testing tools for automation are based on accessing the game objects and interacting with them while running the test script. Some others provide black-box testing based on different techniques.

2.1 The concepts of test automation for mobile games

Today, there is a large range of mobile game testing tools that assist developers and QA teams in optimizing the development process. But before implementing one of those tools into the project, it is important to determine what testing approach is being used and whether it is appropriate for the current project. Because the way the game will be tested is entirely dependent on it. That decision is determined by the developers as well as the type and complexity of the game.

2.1.1 White-box testing

Unit testing is an approach related to game's code context. That is why this kind of testing is a part of white-box testing, which focuses on the architectural and systemic aspects of the mobile game. [13] It requires a thorough understanding of the code prerequisites, programming design, and game engine. It focuses on the mobile games' architectural and systemic aspects. This type of testing focuses only on what is happening within the game. The developers can instantiate the objects and look at how they interact with one another. Although it is just component testing and not the game itself. The developer can still control the scenes and actions but cannot check how the game runs in a real user environment on the real device.

The idea of unit testing is quite popular in software development and testing, but it is not so easy to apply it to a game project. The reason for that is the nature of games, as they have different constraints than most other applications. The main purpose of unit tests is to ensure that the software logic does not change as the result of refactoring and other changes.

Typically, games are being changed a lot, both during production and after release. [13] As a result, the game logic is always inconsistent, as new features are added and there are a lot of changes and improvements to the underlying logic. Therefore, it is hard to maintain them since the developers have to rewrite unit tests many times in order to keep them up to date, so they pass with the new implementation.

Another issue that can arise during unit tests development in game projects is that game functions sometimes do not return any value that can be checked. Instead, they set a process in motion that should be completed at some point in the future. Testing such behavior is worthwhile but requires a significantly different approach. Nevertheless, it is recommended to implement unit tests at the very beginning of the game development process. There are a few frameworks and testing tools on the market that can be implemented into the game project and provide support for unit tests integration.

2.1.2 Gray-box testing

That concept refers to the object recognition approach. Test automation tools must be implemented into the game project before they can access any game object and its properties and interact with them. However, that does not have any impact on the game source, and it interacts with the game elements only within the test script. It can also be called integration testing, as the testers interact with the whole game at a specific point in time when all parts are working, while for unit testing, the inner processes of a game are tested without launching it. Given the fact that test scripts do not depend on resolution, they can be reused on other devices.

Today there is a range of tools for integration testing of mobile games, that allow users to interact with game objects from the IDE inspector and write testing scripts to cover the games' functionality by simulating any kind of device input for user interaction with the game.

2.1.3 Black-box testing

The black-box approach refers to the game testing without any knowledge of what is happening behind the scenes and the main app logic. It is suitable for UI testing, that usually requires a lot of time during manual testing. It covers testing of the visual level of the game display, such as buttons, menus and even some gameplay functionality, if its mechanics are not too complicated for such an approach. However, that might not be proper for games, that have a dynamic interface, such as multiplayer battle arenas, that have an open world and thousands of players interacting with each other. Black-box testing in the games industry works well for simple scenarios. It does not require any changes in the game, and the QA teams can drive the execution 100% from the outside of the game, so it can be tested right from the app store and recreate real users' interactions with the game. [14] It appeared to be good at dealing with such issues as out of memory problems, crashes, and graphic defects. The most popular black-box test automation techniques are coordinate and image recognition.

Coordinate-Based Recognition

That approach is based on defining the exact x and y-axis coordinates on the screen and accessing and interacting with the game's UI elements. [15] One of the problems in the mobile software industry is testing on different resolutions. That approach may cause severe issues while running the tests on various devices, because it will be necessary to update the script for every resolution. The coordinate-based testing does not provide any feedback on whether the script actions were correct or not. There is also no way to interact with game objects or assert anything on the screen. [10]

Image Recognition

The image recognition approach is based on identifying and detecting objects or features on the device screen. It uses different algorithms responsible for image comparison. It allows not only to detect the required elements on the screen but also to do the asserts and check the result. It is a good solution for black-box testing on many resolutions when there is a good algorithm behind it. [16] While using the testing tool based on the image recognition technique the user must take screenshots of the required elements on the device screen and use them in the test script. During the test execution, the tool compares the device screen with the screenshot and tries to reach the defined element.

One of the most popular algorithms for image recognition is based on pixel comparison. It tries to make pixel-perfect matches to look for the exact same image inside a screenshot from the device. Another algorithm is available in OpenCV, that is the most popular open source in the image processing library. It uses a feature and matching detection algorithm called AKAZE, that is resolution agnostic and helps to find an element despite the different elements' angles or rotations. Features in image detection are defined by edges, corners, and other things that the tester tries to identify inside an image when processing it digitally. With the help of AKAZE and OpenCV, it is possible to quickly define and get mobile elements by defining .png files and then building functions that compare the screen and graphics content. [17]

That approach is good for static mobile applications where UI elements are constant and do not change their state during interaction with the app. However, at least half of all mobile games are dynamic and cannot be tested with that technique.

2.2 Limitations of mobile game test automation

It follows from the above that test automation is quite useful in mobile game development, but it has some significant limitations. There are a few factors to consider before implementing test automation into the game development process.

Cost of implementation

Implementing proper test automation might be an expensive and time-consuming process. Due to the complexity of the game mechanics, the game companies must spend a lot of time

on test script development. It may have a significant impact on the design of the code, which is also time-consuming. Furthermore, writing good tests is hard and requires skills and deep knowledge that are not widely available in the game industry. Even though it will pay off in the long term, the game studios must be sure that it will not affect the process of the game development itself, as the main goal is to finish the game to ensure its life span. [18]

The Human Factor

Test automation cannot replace manual testing. Game testing will always need human testers, as automation testing might be inappropriate for some aspects of the game, such as balance, playability, or visual appeal. Another aspect that should be kept in mind is that games require gameplay testing. Even though the main functionality and UI can be tested with automation, there is still a need for the human factor, as only the human eye can understand if the element is in the right place and looking good, or whether a game is well-balanced and fun to play. That should be tested by real people during playtests. [18]

Maintenance

It is rather hard to maintain automation tests due to the flowing state of every game – a lot of updates and changes in UI made by designers or artists. It would be fine as long as the application remained exactly as when the test was scripted. There is a chance that test scripts will not save any time or financial resources, as tests need lifespan specialists' support as they are quite sensitive to any changes and modifications to the tested code. That support might be even more expensive than test realization and implementation.

Tools of-the-shelf

As there are not so many test automation tools for games on the market, the QA may face the problem of implementing them into the game project. Most of the tools are made with graphic cross-platform engines, and it is impossible to access game objects without any additional library or framework. [18] For example, for games made on the Unity engine, any game would be presented as one canvas, and the only option for interacting with the interface is to use coordinated or image recognition approaches, which is not so practical. [19]

3 Game sample prototype development

This chapter describes the design and development of a sample game prototype, which will be used in the following chapter for tool evaluation and will be one of the key measures of the final comparison. The decision to focus on that aspect was made since effective game testing is impossible without the game's source code. The only way to avoid it was to perform black box testing, which seemed ineffective in terms of the gameplay test automation.

3.1 Game design

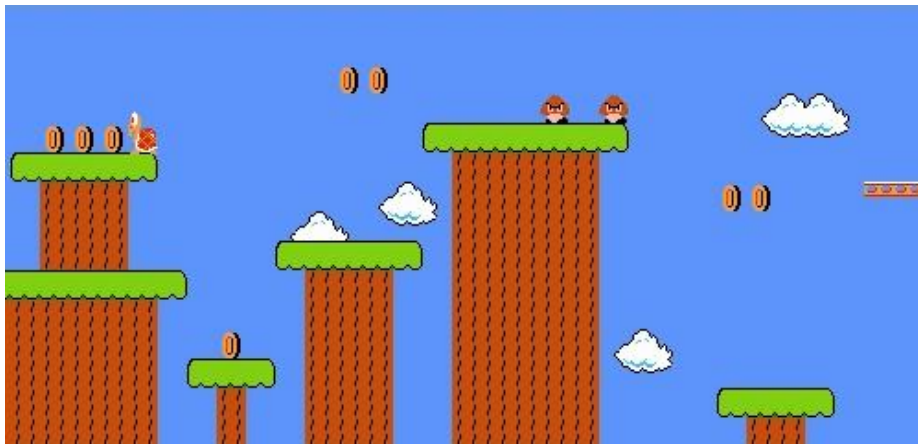
Because game development requires a solid understanding of programming and game engines, the Udemy course "The Ultimate Guide to 2D Mobile Game Development with Unity"¹ was purchased and used. All script templates and assets belong to the owner of the course.

As the course offers the complete process of game development, there was no need to determine the game design and mechanics. After completing the course, the following game's parameters can be determined:

- Game was created with the Unity3D engine, which is one of the most popular game engines among mobile game developers
- C# was used as the programming language because it is Unity's native programming language
- The game is created in the 2D genre, which means that all of the game objects are flat and two-dimensional
- This game is a "platformer", which is one of the most popular game genres for mobile devices. Platform games need the player to run through different barriers and jump on platforms and ledges. In case of the modern mobile games the player is usually controlled with a joystick and buttons.

¹ <https://www.udemy.com/course/the-ultimate-guide-to-mobile-game-development-with-unity/>

Figure 1: Representative of the platform game [22]



Mario is the most well-known representation of the genre. The game mechanics are quite simple and refer to any mobile platform game. They all are discussed in detail for a better understanding:

Environment

- Game has one level, that must be finished
- The player goes back and forward on the surface
- On the level, there are different enemies, that move back and forth at a set interval and react to the players' appearance
- Player can gather coins on the surface. The player must run on the coin to collect it
- Player has a life bar, that can be decreased during a combat

Movement

- Player can run back and forward
- Player can jump. To jump over the crater, the "Jump" button must be pressed when the player is running
- Player can attack
- All coins are gathered automatically

Controls

- The key mechanics of the game are based on using a joystick button for moving back and forward
- "A" button to jump, "B" button to attack

3.2 Test cases

Test cases were proposed for the developed sample game to cover the testing requirements based on gameplay features. Gameplay describes what the player can do in the game as well as what other characters can do in response to the player's actions. No doubt that every game requires structural testing, which includes a menu and its functionality, as well as

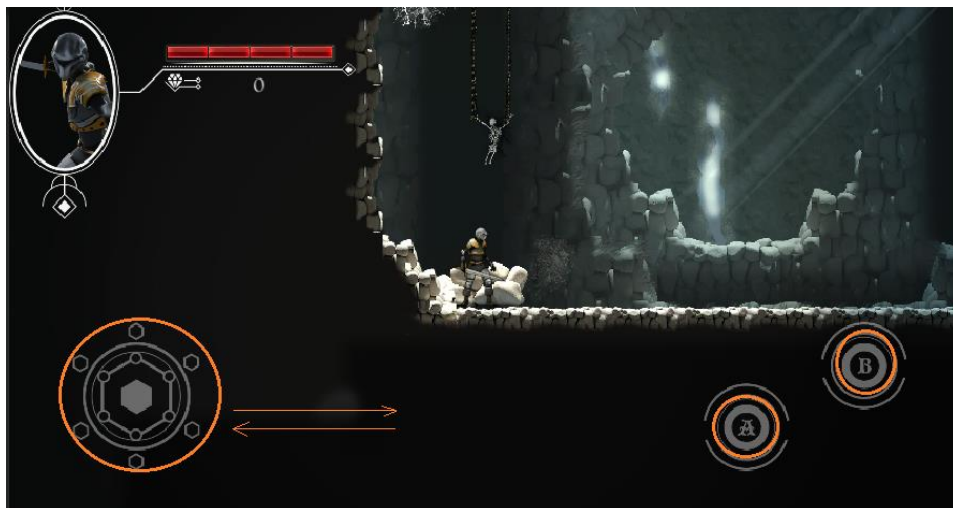
buttons and other controls. However, it can be easily automated and tested, just like any other regular application, while gameplay needs a specific approach. That is why the test scenario will cover all of the game functionalities that were covered in the previous chapter.

It was decided to create simple test suites with test cases based on the game design features described above. A test suite is a combination of related test cases for a particular feature, while a test case is an individual test designed to check a certain feature. [20] It is a list of steps that need to be followed to complete the execution of the test case. After some consideration, the following test suites were created:

Controls

- Test joystick – check whether player moves backwards and forwards
- Test ‘Attack’ button – check whether player starts attacking animation
- Test ‘Jump’ button – whether player starts jumping animation

Figure 2: Game controls



Game mechanics

- The player appears in a predefined position at the start of the level
- The amount of coins increases once the player collects them
- Jump on place
- Jump while moving
- Jump over the crater
- Fight to the spider (damage)
- Fight to skeleton (damage)
- Fight to giant (damage)

Enemies

- Each enemy (Spider, skeleton, giant) moves within the stated interval
- Each enemy reacts to player's appearance and starts an attack
- Each enemy loses life during the player's attack

Figure 3: Game enemies' position



4 Tools overview and definition

This chapter aims to cover the process of selecting the proper automation test tools for mobile game testing. Because game testing tools are not widely available, it was decided to look for any reference of mobile game test automation in forums, blogs, tutorials, and research.

4.1 Selecting of test automation tools

Today, there is a huge demand for automation testing tools, which has resulted in a lot of software testing solutions. All of those frameworks, however, have significant limitations when it comes to interacting with non-native application types like games.

It was decided to look through the academic literature in order to find the best automation test tools for mobile game testing. The following books and reports were used: 'Hands on Mobile App Testing' by Daniel Knott, 'Game Testing All in One' by Charles P. Schults, 'Game QA & Testing' by Luis Levy, 'Automated Tests for Mobile Games: An Experience Report' by Andre Takeshi Endo, 'Game Testing Automation Guidance' by Wael Awad. Due to the lack of information, it was decided to look into alternative less formal sources of information, such as videos and talks from game development conferences ['Selenium Conference Chicago'], blogs, publications, and forums like Reddit and Stack Overflow.

The following criteria were used in order to choose the most relevant sources of information:

- It must be about game development and testing
- The testing tool must be mentioned as one of the options to automate tests
- It must be written or spoken in English

As the result, the following tools were found:

- Appium
- T-plan
- Sikuli
- AirTestIDE
- GameDriver
- AltUnity Tester
- Unity Test Framework

The reason why common testing frameworks like Robotium, Cucumber, Calabash were excluded from that list is their inability to access and interact with the game objects. That is the main disadvantage of regular testing tools. They only support native or hybrid mobile applications, while the majority of games are non-native and built with game engines.

As one of the objectives of this thesis is to define the proper testing tools, suitable not only for regular applications but also for games and the developed game prototype specifically, it was decided to go over each framework and define its testing capabilities.

Appium & T-Plan & Sikuli

After consideration, Appium, Sikuli, and T-Plan frameworks were determined to be insufficient in terms of appropriate UI test automation of mobile games.

Although Appium was originally designed to be a regular testing tool for native or hybrid applications, it may still be used for game testing. However, it is not the best solution, while there is a supply of specifically game testing tools on the market. Appium uses native test frameworks for Android and iOS, that are implemented with native SDK. As previously stated, mobile games are typically built using game engines, that have no relation to native SDK and hence cannot be tested with native test frameworks without additional adjustments. [21] As a result, the user cannot see any game objects in the Appium inspector, which ordinarily reveals the hierarchy of app elements. In the inspector, the only thing that displays is a single canvas with no items to interact with. Although the problem can be eliminated with the help of the AltUnity Tester, that is described further, but it still cannot be used as a stand-alone game testing framework. The identical problem may be seen in the Sikuli and T-Plan frameworks. The only way to test games with the mentioned frameworks is to use coordinates or image recognition techniques.

Although testing with the coordinates appeared to be a bad testing solution. According to Ru Cindrea, who participated in the Selenium Conference 2018, that is irrelevant when the game is intended to be tested on multiple resolutions. [10] Moreover, such testing provides no feedback because the user can only perform actions but cannot see whether they are correct or not. Image recognition, which has previously been discussed in earlier chapters, is also not the best option. The reason for that is that it is not fast enough and does not work for all types of games. As indicated in the Bitbar testing report, it is good for static or slow games, that do not require fast user interactions. [21] So, the tester has enough time to perform the actions, take a screenshot, and process them. However, it is not relevant in terms of dynamic games, where the game is moving, and the user is expected to react quickly.

To summarize, such frameworks are useful for testing fundamental aspects of the game. However, if the aim is to go even further and perform full UI testing of the gameplay and navigation inside the game, this is not the best method.

Unity Test Framework (UTF)

The Unity Test Runner provides support for unit and integration testing in Unity 3D. It allows Unity users to run unit tests with NUnit library integration. [23]

For developers who work in the Unity environment, the Unity Test Framework is a highly valuable testing tool. However, that framework is focused on unit testing only - it allows to instantiate the objects and check how they interact with one another, but it is not for the testing of the game itself. The testers cannot examine how the game is going to run after it

is exported and running on a mobile phone. That is the reason why UTF was not selected for further comparison.

AirTestIDE

AirtestIDE is a cross-engine, open-source UI automation framework, that is designed for "hard-to-automate" applications like games. [24] The AirTest IDE includes two frameworks: AirTest and Poco, and it runs on Windows, MacOS X, and Linux. Airtest is an image recognition framework for mobile games and apps based on image thresholding. It allows interacting with the application using the commands implemented into the framework. Poco is a user interface automation framework that provides APIs independent of the engine, that retrieve the hierarchy of UI elements during the app's running. It gives access to the entire hierarchy of elements.

GameDriver

GameDriver currently supports only the Unity engine. That is a set of libraries that must be installed to the game's build in the Unity editor. That allows GameDriver to connect to the game and get access to any scene that launches within the game while running in the development stage or standalone build. GameDriver's API allows to query object states and provides comprehensive insight into what is going on in the game while test script is running. [25]

AltUnity Tester

This is another open-source UI test automation tool created for Unity. [26] It provides game objects access and can get all their public properties and interact with them. It can simulate any kind of device input, capture screenshots from Unity games, and see test results and reports directly inside the Unity Editor. Tests can be run on real devices or inside the Unity Editor. There is also an opportunity to integrate AltUnity with the Appium testing framework for a better testing experience.

After considering the following, testing frameworks were selected for further comparison: AirTest IDE, GameDriver, and AltUnity Tester. All of these frameworks are considered to be proper for UI driven automation. They support games built with game engines and solve the problem of accessing game objects hierarchy from Unity. Other testing tools were rejected due to the mentioned factors.

5 Automation test tools evaluation

This chapter covers the main goal of this thesis and is divided into several subchapters, that will individually cover the objectives. First, the evaluation parameters were defined. To define the proper criteria in terms of game testing, it was decided to rely on related studies, books, and game testing blogs about automated testing. The defined criteria were divided into the following groups: tool-related, user-friendly-related, test script-related.

The defined criteria were further used for the evaluation of every selected tool by the method of multicriteria analysis, using the ranking method for weight assignment. According to the assigned weights and scale, the selected tools were evaluated and compared. Additionally, the test scripts covering the defined test cases were created within every selected tool and then commented with regards to the defined criteria. Finally, the most convenient tool in terms of mobile game test automation was recommended.

5.1 Defining evaluation criteria

This subchapter covers one of the objectives of the thesis and defines the criteria for accomplishing the main thesis goal: tool comparison. As it was already mentioned in the thesis, the tool comparison and testing itself is focused on functional game testing only. As the main difference between a game and regular app testing is the game's dynamic objects and unpredictable users' behaviour, it was decided to focus on gameplay testing and compare those tools' features, that have a direct impact on it. No doubt that any game needs structural testing, which covers not only gameplay but also the menu, game store testing. However, it can be easily automated and tested, as with any regular application, while gameplay needs a specific approach.

When defining the proper criteria for mobile game testing, the following major points were taken into account:

1. The testing tool developers' position on automated game testing— it was decided to go through all their interviews and blogs. The following inquiries were made:
 - What was their intention in developing the technology for automating mobile game testing? [27]
 - What significant characteristics of the tools were highlighted by the developers?
2. Criteria defined for regular app testing tools. [28] [29] It was also determined to go through the list of the most crucial mobile app frameworks' characteristics. The majority of the functionality of mobile applications are carried over into mobile games. Book 'Hands on mobile app testing' by Daniel Knott was used as the main source of the information. [15]
3. Studies and reports on related topics.

It was determined to divide the criteria into three categories: tools and technical requirements, user-friendly requirements, and test script requirements. All of the criteria have distinct functional requirements, which is why each one includes an assessment description and a scale of 1–10, with 1 indicating the least good and 5 indicating the most satisfactory. The values are determined by the needs of each specific criterion.

The first set of criteria relates to the technical aspects of the testing tools that have been chosen. C1-C5 are connected to the testing tools' ability to interact with the game as well as the specific options for test script development, which must deliver a better test automation experience. The following criteria C6-C8 outline the testing alternatives provided by the testing tools, which expand game developers' abilities to test games from various viewpoints and separate the game and test logic in order to make them independent of one other. The remaining criteria in the current set refer to low-priority features of testing tools that may be useful in terms of game test automation but are not important.

Table 1: Set of the tool related criteria [15] [28]

ID	Criteria name (description)	Evaluation scale
C1	Mobile gestures: • <i>Tap</i> – quick interaction, briefly touch the screen with a fingertip • <i>Double tap</i> – rapidly touch surface twice with fingertip • <i>Touch</i> – touch the screen for a longer time than tap • <i>Swipe</i> – slide finger across the surface without losing contact [30]	1 – does not provide any mobile gestures for testing 5 – does not provide all required gestures, but this has no effect on the testing process 10 – all gestures are supported
	The ability of the tool to imitate any form of mobile device input. This parameter was specified in relation to any mobile testing framework. Mobile games inherit standard app features and, as a result, require the same basic testing option in testing frameworks.	
C2	Find game objects and interact with them from a script	1 – cannot access any game object 5 – can query the limited number of objects and properties 10 – can query any game object, access its parameters, modify them.
	Query game elements to get their properties, use and modify them. It is specific to game testing. One of the most significant issues that game developers experience during test automation is the inability to access game objects from the testing tool's inspector. [10]	

C3	Capture the current running FPS	1 — does not provide FPS capturing 10 — provides FPS capturing
	It is specific to game testing. The number of FPS in a game is determined by the device, internet speed, and game adaptability. One of the most important aspects of game testing is to run it on a variety of resolutions and devices. If a duration time specification is required, it may be different for each device.	
C4	Allows delays in testing scripts	1 — does not provide any test delays 10 — provides test delays
	It is related to any mobile testing framework and is directly related to the C3. It helps you to manage the testing process and the time it takes to execute a method.	
C5	Gets screenshots from the game	1 — cannot capture screenshots 10 — can capture screenshots
	It refers to the common mobile app testing. The tool must have the ability to make a screenshot to ensure that the correct elements are seen on the screen.	
C6	Allows running tests in edit and play mode: <i>Edit mode</i> – run tests in IDE during the development <i>Play mode</i> – allow to exercise the actual game simulation in real and verify that it runs as desired.	1 — tests can be executed only in one mode 10 — tests can be executed in both modes
	It is specifically for game testing. Both functions must be supported by testing tools. It is vital to test the game during development to verify that no logic is violated as new features are added. Play mode is intended to address the issue of testing the game on various platforms and get user feedback.	
C7	Environment for testing	1 — runs tests only within the IDE 10 — allows testing in the separate inspector

	It may be a good practise to keep the development and testing environments separate. There is less possibility that the code will be corrupted by testers if the game is explored and reviewed outside of the development environment without access to the source code.	
C8	Supports different platforms	1 — supports only Unity3D 10 — supports different game engines
	That property is specifically for games. There is a huge range of game engines that game developers use. Unity 3D is one of the most popular. Some developers, however, choose different engines, which must also be supported by testing frameworks.	
C9	Supports record and replay property	1 — does not support property 10 — supports property
	This feature allows the tester's actions on the game screen to be recorded into the script. Those scripted actions can later be used to generate tests. This is helpful when developers are unfamiliar with the testing tool environment and test format. This can also assist testers who do not have extensive coding experience in obtaining the test foundation for future upgrades.	
C10	Has additional integrations for testing	1 — does not provide additional features 10 — offers additional integration
	That parameter has a low priority since the main tool's capability must be sufficient for appropriate test automation and execution. However, it is beneficial to incorporate certain extra features so that each game has a better chance of being tested successfully, as each one requires a distinct strategy.	

This set of criteria refers to the user experience gained via the using the test automation tools. Some of those criteria are based on related research on mobile app testing, while others are based on the author's own experience.

Table 2: Set of the user-friendly related criteria

ID	Criteria	Evaluation scale
C11	Requires awareness about the game code and programming skills	1 — requires game code knowledge and high coding experience 5 — has easy test syntax but requires some coding experience 10 — does not require game code knowledge and is easy in test development

	Can the individuals whose job is to do manual testing and that do not have solid knowledge in programming.	
C12	Fast and easy to setup working environment	1 — challenging setup 5 — requires additional actions for setup 10 — easy installation and setup
C13	Documentation available	1 — does not provide any documentation or support 5 — documentation does not provide all necessary information 10 — provides updated documentation available for everyone. Provides technical support.
	In any framework or application, documentation is critical. It must include all required information regarding tool use, such as installation instructions and examples of how to use the tool. Furthermore, technical assistance is required.	
C14	License price	1 — close source 5 — license is paid, but provides a free trial period 10 — open source
C15	Script languages	1 — supports one script language, that is hard to integrate without experience 5 — supports one script language, that does not require deep knowledge 10 — supports multiple languages, including those that are suitable for people without deep coding knowledge

The test script implementation aspect (criteria C16-C20) refers to the test code development. Test scripts will cover test suites with test cases that were previously defined in the previous chapter. The test suites for the sample game prototype were implemented using each of the tools, AltUnity Tester, AirTestIDE, and GameDriver, and discussed in the next chapter.

Table 3: Set of the test script related criteria

C16	Test scripts execution time	1 — long-time of execution 5 — exceptional time of execution 10 — fast execution without any delays
C17	Test report generation	1 — does not have any log repots 5 — does not provide full log report 10 — step by step report, provides structured reports
C18	Covered test cases	1 — under 60 % of all test cases 5 — between 60-80 % of all test cases 10 — over 90 % of all test cases
C19	Test stability	1 — less than 80 % of test cases are stable 5 — between 80-90 % of test cases are stable 10 — 100 % of test cases are stable and do not fail

5.2 Assign criteria weights using a ranking method

The next step is to calculate the criteria weight. To assign the criteria weights it was decided to use the ranking method, which is one of the common weighting methods used in multicriteria decision analysis. The weights were calculated using the rank sum method, in which the weights are computed from the individual ranks normalized by dividing the sum of the ranks. [31]

The criteria below were rated on a scale of 1-10 according to the priority for mobile game testing automation. A number of 10 indicates the highest priority, while a value of 1 indicates the lowest. The individual criteria's weights were then calculated using the scoring method, which involved dividing each value by column total of all points allocated to all criteria. That table will be used in the following subchapter for tool evaluation, where each weight will be multiplied by the evaluation score of each individual tool.

Table 4: Calculation of the weights of individual criteria.

Criteria	Rank	Weight
C1	10	0,08849558
C2	10	0,08849558
C3	5	0,04424779
C4	3	0,02654867
C5	4	0,03539823
C6	8	0,07079646
C7	4	0,03539823
C8	6	0,05309735
C9	3	0,02654867
C10	2	0,01769912
C11	5	0,04424779
C12	3	0,02654867
C13	8	0,07079646
C13	7	0,0619469
C15	4	0,03539823
C16	3	0,02654867
C17	5	0,04424779
C18	10	0,08849558
C19	3	0,02654867
C20	10	0,08849558
Total	113	1

5.3 AltUnity Tester

This subchapter describes the process of AltUnity Tester evaluation based on defined criteria and multicriteria analysis. It also provides the comments on the developed test scripts and describes the limitations of the framework.

5.3.1 AltUnity Tester evaluation

The tool is easy to implement, and it does not require any additional steps except importing the files from the Unity Asset Store and creating the testing scripts within the specific folder. The tool allows running tests in the Unity Editor as well as on mobile devices or simulators. AltUnity Tester provides detailed documentation support based on tracking the issues on GitLab. It also offers separate paid inspector for executing tests without access to the Unity Editor.

One of the most important features of the tool is reaching the game elements and its public properties, like coordinates, text, or values. The tool allows capturing any elements on the scene, no matter whether they are visible on the game screen. However, the author is faced with the issue of reaching the object properties that are inherited from other classes and contain other unity components. No information regarding that was found in the tool documentation. That slowed down the script development but was not critical. Besides AltUnity Tester offers the possibility of integrating Appium to interact with native elements.

Overall, tool proved itself to be effective in terms of mobile game testing. It has good user interface and allows controlling the test flow and running them separately on any game scene. All test cases were automated successfully, but 2 of them were not stable and failed from time to time.

Table 5: Evaluation of AltUnity Tester.

Criteria	Evaluation	Weight	Result
C1	10	0,0884956	0,8849558
C2	8	0,0884956	0,7079646
C3	1	0,0442478	0,0442478
C4	5	0,0265487	0,1327434
C5	10	0,0353982	0,3539823
C6	10	0,0707965	0,7079646
C7	5	0,0353982	0,1769912
C8	1	0,0530973	0,0530973
C9	1	0,0265487	0,0265487
C10	10	0,0176991	0,1769912
C11	6	0,0442478	0,2654867
C12	10	0,0265487	0,2654867
C13	10	0,0707965	0,7079646
C14	8	0,0619469	0,4955752
C15	10	0,0353982	0,3539823
C16	10	0,0265487	0,2654867
C17	5	0,0442478	0,2212389
C18	10	0,0884956	0,8849558
C19	6	0,0265487	0,159292
C20	8	0,0884956	0,7079646
Total	138	1	7,433628319

5.3.2 Implementation and testing

The test script was written in C# language, as it is a native language for Unity development and author decided to continue using it after developing sample game prototype. The test script development process was fast and easy due to the comprehensive documentation, covering all the features. It was decided to combine 'Game Mechanics' test suit with some test cases from 'Enemies' test suit, so they can be checked during the general level testing. All test cases in that group determine key features of the game.

There was no failure in the executed test suites. However, 2 test cases failed once in while for some unknown reason. Execution of the test suit 'Enemies' which is responsible for defining the enemies' position, varied within 24-29 seconds and was completed with success. Execution of the test suit 'Controls' which checks joystick and buttons functionality, lasted 8,7689167 seconds with success. Execution of the test suit 'Game Mechanics' which represent the whole level testing, was the longest one and lasted 39,6750578 seconds with success. Total time to execute test cases is 73,26562 seconds.

Table 6: AltUnity Tester test cases automation.

Test Suit	Test Case	Test status	Execution time
Controls	Joystick	Success	8,7689167 seconds
	'Attack' Button	Success	
	'Jump' Button	Success	
Enemies	Movement interval	Success	29.8043994 seconds
Game Mechanics (Level Testing)	Player starting position	Success	39,6750578 seconds
	Jump on place	Success	
	Jump over crater	Success	
	Jump while moving	Success	
	Fight to spider	Success	
	Fight to skeleton	Success	
	Fight to moss giant	Success	
	Deals damage to player	Success	
	Gets damage from player	Success	
	Coins behaviour	Success	

AltUnity Tester enables a proper interaction with the game objects, that was the key point to successful sample game testing. The tool allows reaching the game objects by their name or tag [Code 1]. That is a fundamental feature of the tool that is responsible for all further test interactions with the game.

Code 1: AltUnity Tester script (access game objects)

```

1. var player = altUnityDriver.FindObject(By.NAME, "Player");
2. var gCurrentPosition = giant.GetComponentProperty("UnityEngine.Transform",
   "position.x");
3. altUnityDriver.FindObject(By.NAME, "BBtn").Tap();

```

The code below [Code 2] is one of the methods simulating the player running on the screen and it is responsible for jumping over the craters at certain points at the scene. As the testing tool is based on interacting with the game objects there was no way to optimise the whole process of running, to make the player jump before any crater. The reason is that no game objects define the starting point of the crater. One of the ways was to add an invisible game object before each crater, however, the purpose of the testing and comparison was to see whether the testing tool can test any game mechanics without interfering into its' logic. That is why each craters' coordinate on the x-axis was chosen as the key x coordinate, where the player must jump.

Code 2: AltUnity Tester script (game mechanics test case)

```
1. void runJump(int jump_x)
2. {
3.     int initialPosition = altUnityDriver.BeginTouch(new AltUnityVector2(joystick.x +
4.     10, joystick.y + 70));
5.     AltUnityVector2 nPos = new AltUnityVector2(joystick.x + 200, joystick.y + 70);
6.     while (convertPosition(player.GetComponentProperty("UnityEngine.Transform",
7.     "position.x")) < jump_x)
8.     {
9.         altUnityDriver.MoveTouch(inPos, nPos);
10.    }
11.    altUnityDriver.FindObject(By.NAME, "BBtn").Tap();
12.    Thread.Sleep(2000);
13.    altUnityDriver.EndTouch(inPos);
14. }
```

The most decent test case is created for testing the enemy's movement within the stated points on the scene. Due to the inability of the framework to reach some object properties it was impossible to get the starting and ending coordinates of each enemy.

Code 3: AltUnity Tester script (enemy test case)

```
1. void checkPointA(String currentPosition, String aPosition, AltUnityObject enemy){
2.     while (convertPosition(currentPosition) != convertPosition(aPosition))
3.     {
4.         currentPosition = enemy.GetComponentProperty("UnityEngine.Transform",
5.         "position.x");
6.     }
7.     Assert.AreEqual(convertPosition(currentPosition), convertPosition(aPosition));
8.     Thread.Sleep(500);
9.     Assert.AreNotEqual(convertPosition(currentPosition), convertPosition(aPosition) - 1);
10. }
```

The function above is responsible for determining the NPC's location on the game scene. Each enemy has two fixed points of the x-axis that restrict their movements. The method checks whether the NPC reached one of the points and did not move further. That test cases requires more time, as it waits until each enemy reaches both points in the scene.

5.4 AirTestIDE

This subchapter covers the evaluation of AirTestIDE and provides the results of the test suites automation.

5.4.1 AirTestIDE evaluation

As it was already mentioned AirTestIDE provides two frameworks for game testing. One of them is Poco, that is going to be used for comparison and test automation purposes. The second framework is AirTest that is not used in that comparison due to the image recognition approach. Poco supports several game engines, what distinguish it from other testing tools, that are focused only on Unity3D. It also supports the script development in different programming languages. Python is used as the default programming language,

that might be a good beginning for manual testers, who decided to implement test automation into the routine.

The framework has easy implementation, however, requires some additional steps as importing the tool package and adding special script to the game object. The authors of the tool provide comprehensive documentation, that covers a lot of projects' and testing aspects. The project also has big community and blog with additional articles regarding the testing. It also offers the possibility to raise and follow issues on GitHub. The tool allows running in both Unity Editor and real devices or simulator. It also offers its own mobile farm for testing on multiple devices.

Unlike AltUnity Tester Poco can only access objects, that are visible on the game screen. Some test cases, therefore, could not be tested. As a result, the whole process of the level testing is based on time settings, that is not the reliable approach.

Table 7: Evaluation of AirTestIDE.

Criteria	Evaluation	Weight	Result
C1	7	0,0884956	0,619469
C2	5	0,0884956	0,4424779
C3	1	0,0442478	0,0442478
C4	10	0,0265487	0,2654867
C5	7	0,0353982	0,2477876
C6	10	0,0707965	0,7079646
C7	10	0,0353982	0,3539823
C8	10	0,0530973	0,5309735
C9	7	0,0265487	0,1858407
C10	1	0,0176991	0,0176991
C11	8	0,0442478	0,3539823
C12	8	0,0265487	0,2123894
C13	10	0,0707965	0,7079646
C14	10	0,0619469	0,619469
C15	10	0,0353982	0,3539823
C16	10	0,0265487	0,2654867
C17	10	0,0442478	0,4424779
C18	5	0,0884956	0,4424779
C19	5	0,0884956	0,4424779
Total	137	1	7,2566372

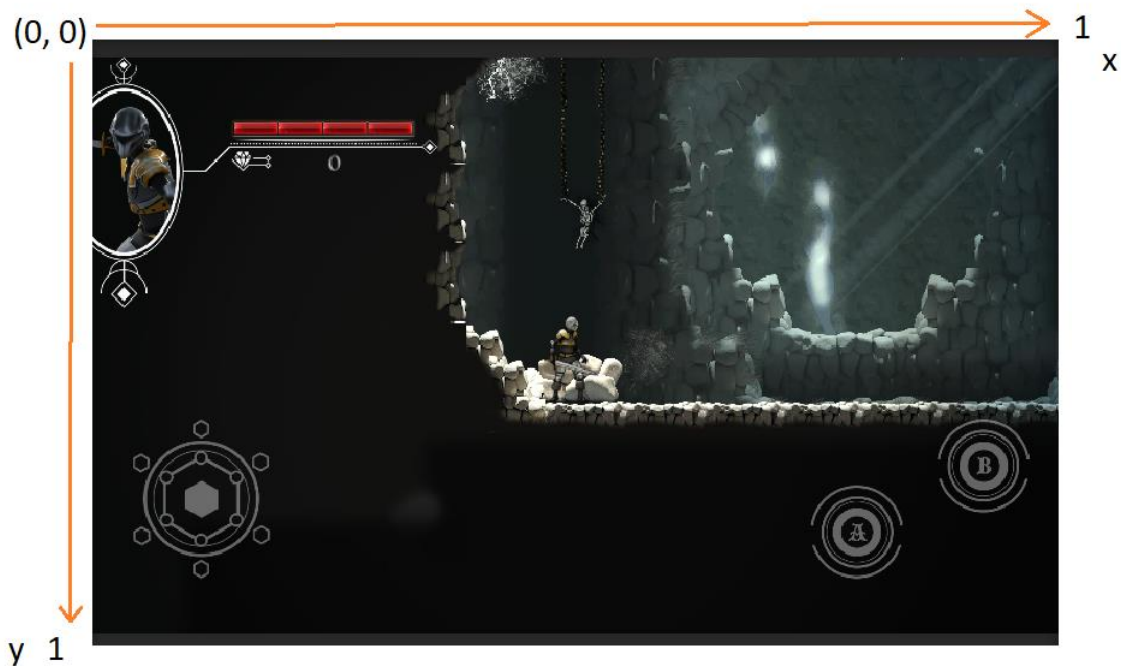
5.4.2 Implementation and testing

AirTestIDE had some limitations that did not allow to automate all the test cases. The main issue was inability of the framework to get required object position. Instead of getting its'

position on the game scene the framework captures it on the screen within $[1,1]$ coordinates. As the result, the test script cannot access items that have not yet displayed on the screen.

Player always stays in the middle coordinates $[0.5,0.5]$ of the screen, that does not allow to test his changing position while he is running or jumping. That may also cause the unpredictable test behaviour, as it is impossible to define the exact objects' position on the scene, as it is always dependent on the player's position. Another issue that occurred is inability to produce two actions at the same time, that is required for jumping over the crater.

Figure 4: AirTestIDE coordinates system



Because of the issues mentioned above, some test cases were not automated. 9 of the test iterations passed, while 5 iterations failed. Some of the successfully automated test cases are not reliable, as they cannot extract the objects information from the game runtime. So, they do not test the exact behavior, but only the outcome. Test cases as 'Attack' button and 'Jump' button are tested by capturing the screenshot to check the correct animation.

Table 8: AirTestIDE test cases automation.

Test Suit	Test Case	Test status	Execution time
Controls	Joystick	Success	25.523 seconds
	'Attack' Button	Success	
	'Jump' Button	Success	
Enemies	Movement interval	Fail	0 seconds
Game Mechanics (Level Testing)	Player's starting position	Fail	73,917 seconds
	Jump on place	Success	
	Jump over crater	Fail	
	Jump while moving	Fail	
	Fight to spider	Success	
	Fight to skeleton	Success	
	Fight to moss giant	Success	
	Deals damage to player	Fail	
	Enemies' life decrease	Success	
	Coins behaviour	Success	

The code below finds the button elements from the unity UI tree and taps them to simulate the jump and attack actions. In order to verify, that the test was successful the screenshots are made, that must capture the players animation. However, sometimes they are not able to capture the required animation, as the player moves too fast.

Code 4: AirTestIDE script (access game objects)

```

1. poco("B_Btn").click()
2. snapshot()
3. time.sleep(1.0)
4. poco("A_Btn").click()
5. snapshot()

```

The following function [Figure 5] is responsible for simulating the attack actions to fight with the enemies. Poco does not enable getting unique element properties as enemies health, so the code checks whether the enemy still exists on the screen and UI tree.

Code 5: AirTestIDE script (attack test case)

```

1. def attack(enemy):
2.     while(enemy.exists()):
3.         time.sleep(0.2)
4.         poco("A_Btn").click()

```

Due to the framework's limitations the different testing approach was used for some test cases as reaching an enemy or simulating player's run. As the game starts, there is no spider on the screen, so the test script cannot capture it and set the destination point for the player. It was decided to let the player run forward until the spider appears on the screen and then continue running as long as both characters are on the same position.

Code 6: AirTestIDE script (mechanics test case)

```
1. while (get_x(poco("Sprite").get_position()) <
    get_x(poco("Spider_Idle_0").get_position())):
2.     joystick.swipe('right', duration = 0.01)
```

The test above is unreliable and cannot be run on both mobile device and Unity Editor. As they require slightly different time settings.

5.5 GameDriver

The chapter covers the evaluation of the last automation test framework – GameDriver.

5.5.1 GameDriver evaluation

GameDriver is an API that must be implemented into the game source code in the Unity editor, so it acts as the agent to the API calls and can control the game. That allows GameDriver to connect into the game and get an access to any scene that launches within the game while running in the development stage or standalone build. GameDriver's API allows to query object states, giving complete visibility into what is happening in the game during test runtime. It supports mobile game testing for Android and iOS and allows test application builds for both devices and Unity editor.

GameDriver does not provide free license of their API to all the users, however, it is possible to request the trial license for a month to get know the API and make a decision regarding its' usability for the current project. Authors provide the documentation, that covers all methods that can be used in the script, and also examples and installation tutorials. Besides, it has an ability to record actions in the game, that can be used as a base for the script development.

The tool provides a wide range of options for interaction with game objects. Furthermore, it has the functionality, that enables to create the reusable tests that can be ran on different devices and resolutions — capturing current running FPS, changing timescale used for test execution. Above all the tool was extended with the support for Unity Test Framework, that improves test integration into the project and allows manage automated test inside the Unity project.

Table 9: Evaluation of GameDriver.

Criteria	Evaluation	Weight	Result
C1	10	0,08849558	0,88495575
C2	8	0,08849558	0,7079646
C3	10	0,04424779	0,44247788
C4	10	0,02654867	0,26548673
C5	10	0,03539823	0,3539823
C6	10	0,07079646	0,7079646
C7	1	0,03539823	0,03539823
C8	2	0,05309735	0,10619469
C9	10	0,02654867	0,26548673
C10	10	0,01769912	0,17699115
C11	6	0,04424779	0,26548673
C12	8	0,02654867	0,21238938
C13	6	0,07079646	0,42477876
C14	4	0,0619469	0,24778761
C15	4	0,03539823	0,14159292
C16	10	0,02654867	0,26548673
C17	10	0,04424779	0,44247788
C18	10	0,08849558	0,88495575
C19	10	0,08849558	0,88495575
Total	139	1	7,716814

5.5.2 Implementation and testing

GameDriver uses C# programming language, that is also used in the Unity3D. It was decided to use Visual Studio, as it was already integrated with the Unity. The script test is based on NUnit framework. GameDriver supports a few testing options – running the test from the IDE or from the game executable. Otherwise, the test script will connect to the Unity editor and initiate Play mode.

GameDriver provides a wide range of methods for interacting with the game objects. To access the element the relevant path from the Unity UI tree is required. Although limited object properties can be retrieved. Currently API has a support for properties with the serializations, as string, int, float, bool.

Despite the complex setup and the hard scripting language (that might be hard for beginners) GameDriver showed excellent results during the test suits automation. All test cases were covered and successfully completed. The test scripts made in GameDriver are similar to the AltUnity Tester's, nevertheless, it was easier to work with the coordinates and object positions, as GameDriver enables representation of 2D and 3D vectors, that does not require any additional number conversion.

Table 10: GameDriver test cases automation.

Test Suit	Test Case	Test status	Execution time
Controls	Joystick	Success	7, 89625 seconds
	'Attack' Button	Success	
	'Jump' Button	Success	
Enemies	Movement interval	Success	26,40139 seconds
Game Mechanics (Level Testing)	Player's starting position	Success	36,77601 seconds
	Jump on place	Success	
	Jump over crater	Success	
	Jump while moving	Success	
	Fight to spider	Success	
	Fight to skeleton	Success	
	Fight to moss giant	Success	
	Deals damage to player	Success	
	Life decrease	Success	
	Coins behaviour	Success	

The test suites' execution time is not significantly different from AltUnity Tester. GameDriver, on the other hand, produced faster results since certain of the functions were carried out more precisely.

The object identification is a core capability of GameDriver, and it is used throughout the whole test script. It uses HierarchyPath to identify objects and their properties' values.

Code 7: GameDriver script (access game objects)

```

1. string player = "//*[@name='Player']";
2. int healthSpider =
   api.GetObjectFieldValue<int>("//*[@name='Spider']/fn:component('Spider')/@Health");
3. Vector3 joystick =
   api.GetObjectPosition("//*[@Untagged[@name='Canvas']/Untagged[@name='HUD']/Untagged[@name='JoyStick']");

```

GameDriver does not have the same touch input implementation as AltUnity Tester, that produces the joystick's imitation by start and stop events. Instead, the number of frames is used to determine the length of the input. For better test automation GameDriver allows to capture current FPS, which depends on the performance of the machine running the game. That allows to use the same code on many devices. The function below uses game engine's FPS to determine the time of the touch event. One of the test cases is to run and jump over the crater. In order to target the player to move towards, the x-coordinates must be defined. The start of each crater must be set as a parameter in the bellow function. The player will keep moving to the right and after reaching the left-side of the crater the jump button will be pressed for 1 second.

Code 8: GameDriver script (mechanics test case)

```

1. void runJump(int x_jump)
2. {
3.     while (api.GetObjectPosition("/Player[@name='Player']").x <= x_jump)
4.     {
5.         api.TouchInput(new Vector2(joystick.x, joystick.y), new Vector2(joystick.x + 200,
joystick.y), 0, 1, (ulong)api.GetLastFPS());
6.         api.Wait(300);
7.     }
8.     api.TouchInput(new Vector2(joystick.x, joystick.y), new Vector2(joystick.x + 200,
joystick.y), 0, 1, (ulong)api.GetLastFPS());
9.     api.Wait(300);
10.    api.TapObject(bBtn, 1, (ulong)api.GetLastFPS());
11. }

```

Script for testing enemies' position is similar to the previous examples. However, the test case has less number of lines, as GameDriver has easier way of interacting with game objects' properties and position.

Code 9: GameDriver script (enemy test case)

```

1. void checkPointA(float currentPosition, float aPoint, string enemy)
2. {
3.     while (currentPosition != aPoint)
4.     {
5.         currentPosition = api.GetObjectPosition(enemy).x;
6.     }
7.     Assert.AreEqual(currentPosition, skeletonPointA);
8.     api.Wait(500);
9.     Assert.AreNotEqual(currentPosition, aPoint - 1);
10. }

```

5.6 Results of the evaluation

Comparison and evaluation of the test automation tools for mobile games fulfilled the main goal of the thesis. The results of the evaluation are summarized in the Table 11, where are also sorted according to the final evaluation.

Table 11: Results of the evaluation.

Nº	Testing tool	Evaluation
1	GameDriver	7,716814
2	AltUnity Tester	7,4336283
3	AirTestIDE	7,2566372

The tool comparison was based on a multicriteria analysis and specific criteria definition. The criteria weights were determined and calculated according to the ranking method with the sum approach. Based on the evaluation results, it can be concluded that GameDriver is the tool, that meets the greatest number of criteria and has the highest evaluation score,

whereas AirTestIDE got the lowest score. None of the tools matched all of the requirements, as each of them focuses on different aspects, such as better user interface or more technical capabilities. Each tool has specific limitations in different aspects, that were initially divided into the criteria sets. Each of the three tools meets the standards for typical mobile app testing, which means that each tool is capable of producing mobile motions in the script and providing basic test coverage to any game or application. However, the game-specific criteria were evaluated differently.

AirTestIDE proved itself as a user-friendly testing framework, that offers the separate inspector and enables a variety of interactions with the device. It is the only framework that support not only Unity, but also other game engines which is important for game developers that deal with different technologies. That is why certain criteria were given a higher score than others, indicating that the framework delivers an excellent user experience. Despite the fact that AirTestIDE's final evaluation score was fairly near to other tools, it did not show itself to be a suitable tool for testing the sample game prototype. It has the limitations related to capturing some of the objects and their properties, that affected the final score. As the result it did not show satisfactory results during test script development since it did not cover all of the test cases. In summary. out of 14 test iterations that the test automation aimed for, 9 test iterations passed, while 5 test iterations failed.

Overall AirTestIDE is good for testing deterministic games such as puzzles or cards, where there are less replay variations and much simpler steps to produce the desired outcome. However, it was inefficient in terms of active platform runner since it required more effort and better object recognition.

Better testing opportunities are provided by GameDriver and AltUnity Tester. These testing tools are the best option for game test automation according to the comparison analysis. There were no significant differences between those two tools, so they both got very close evaluation scores. These tools are pretty good in interacting with game objects, however, they both have one significant disadvantage — the inability to access some of the game objects' properties. Both tools can be implemented only to Unity3D. Although GameDriver is an independent project, that is going to be extended by additional support for other game engines. Nevertheless, the comparison shows that GameDriver is few steps forward from the AltUnity Tester.

In order to compare the test stability of the tools, the test cases were executed several times using each of the three tools. AirTestIDE and AltUnity Tester showed unstable results on some test cases, while GameDriver's test were completed successfully during each execution. GameDriver also showed the better results in terms of tests' reusability on different devices, as the test cases are mostly based on FPS capturing instead of time settings. That allows to optimize the tests and reuse them on any running machine. results and ran the tests in a very short time.

Overall GameDriver is a great choice for mobile game testing. It provides a comprehensive set of features for interacting with the game and implementing full automated testing solutions for the project. Besides it enables the record and playback functionality, that helps to understand how the API works and use the recorded functions for further script development. Even though the license is paid, it is still worth it.

Conclusions

This bachelor's thesis covered some fundamental theory about mobile game testing as well as potential challenges that testers may face during the test implementation. The thesis also went through the various test automation options and described each of them.

The main goal of the thesis was to define the relevant testing tools for mobile game testing and compare them for the purpose of determining a suitable tool for games' tests automation. After the review of the available literature three test automation tools (GameDriver, AitTestIDE and AltUnity Tester) were chosen and consequently evaluated. The comparison was based on multicriteria analysis of defined criteria. By defining the criteria, the author attempted to address all significant areas of mobile game testing that have an influence on the test implementation into the game project. The results show that while one tool rated better than the other for some comparison criteria, it ranked worse for others. That is why there was no significant difference in the final evaluation score. Nonetheless, GameDriver was evaluated as the most relevant solution for test implementation.

To summarize, there is no one-size-fits-all method for game testing since each game requires a unique approach. Implementation of the test automation allows to reduce the amount of time spent on manual regression testing. However, it is important to remember that the manual testing will never be fully replaced with anything and none of the testing approaches will provide the 100 % result. The most efficient way is to combine both manual and automated testing. Before implementing any testing tool into the game project, it is highly recommended to explore a variety of tools in order to make the right decision and choose the tool that best suits the demands of your project.

List of references

- [1] MAIDA, J., 2020. Gaming market size to grow over \$119 billion at 12 % CAGR by 2024: Technavio [online]. [Accessed 8 December 2021]. Available from: <https://www.businesswire.com/news/home/20201106005457/en/Gaming-Market-Size-to-Grow-Over-119-Billion-at-12-CAGR-by-2024-Technavio>
- [2] NEWZOO, 2021. Newzoo Global Games Market Report 2020: Light version [online]. [Accessed 8 December 2021]. Available from: <https://newzoo.com/insights/trend-reports/newzoo-global-games-market-report-2020-light-version/>
- [3] SCULTZ, Charles P, LANGDELL, Tim and BRYANT, Robert. Game testing all in one. Course Technology, 2005. ISBN 1592003737.
- [4] NYMAN, J., 2013. Testing Games is Hard! [online]. [Accessed 8 December 2021]. Available from: <http://testerstories.com/2013/08/testing-games-is-hard/>
- [5] HOBERG, J., 2014. Differences between Software Testing and Game Testing [online]. [Accessed 8 December 2021]. Available from: <https://www.gamedeveloper.com/programming/differences-between-software-testing-and-game-testing>
- [6] BRETT ROARK, 2014. Test Automation Challenges in the Gaming Industry [online]. [Accessed 8 December 2021]. Available from: <https://www.slideshare.net/TechWellPresentations/t6-30218687>
- [7] CALIFORNIA NEWS TIMES, 2021. Challenges of game testing [online]. [Accessed 8 December 2021]. Available from: <https://californianewstimes.com/challenges-of-game-testing/522651/>
- [8] JOSHI, M., 2019. A case study on Game Automation Testing [online]. [Accessed 8 December 2021]. Available from: <https://blog.yudiz.com/a-case-study-on-game-automation-testing/>
- [9] QA MADNESS, 2020. Mobile Game Testing for Android and IOS: Checklist & types of testing [online]. [Accessed 8 December 2021]. Available from: <https://www.qamadness.com/mobile-game-testing-for-android-and-ios-checklist-peculiarities-types-of-testing/>
- [10] CINDREA, R., 2019. SeleniumConf Chicago. In: Youtube [online]. [Accessed 8 December 2021]. Available from: https://www.youtube.com/watch?v=hSn6qwh6b28&t=37s&ab_channel=SeleniumConference
- [11] SOFTWARE TESTING HELP, 2021. What is Automation Testing (ultimate guide to start test automation) [online]. [Accessed 8 December 2021]. Available from: <https://www.softwaretestinghelp.com/automation-testing-tutorial-1/>

- [12] VILLE-VEIKKO HELPPI, 2015. The Basics Of Test Automation For Apps, Games And The Mobile Web [online]. [Accessed 8 December 2021]. Available from: <https://www.smashingmagazine.com/2015/01/basic-test-automation-for-apps-games-and-mobile-web/>
- [13] HAMILTON, T., 2021. Game Testing: Types & How to Test Mobile/Desktop Apps [online]. [Accessed 8 December 2021]. Available from: <https://www.guru99.com/game-testing-mobile-desktop-apps.html>
- [14] TOY TESTING. Types of Game Testing Techniques [online]. [Accessed 8 December 2021]. Available from: <https://toy-testing.org/testing-techniques.html>
- [15] KNOTT, D. Hands-On Mobile App Testing. Addison-Wesley, 2015. ISBN 0134191714.
- [16] MEEL, V., 2021. Image Recognition: The Basics and Use Cases (2022 Guide) [online]. [Accessed 8 December 2021]. Available from: <https://viso.ai/computer-vision/image-recognition/>
- [17] HELPPI, V., 2020. Using OpenCV and Akaze for Mobile App and Game Testing [online]. [Accessed 8 December 2021]. Available from: <https://bitbar.com/blog/using-opencv-and-akaze-for-mobile-app-and-game-testing/>
- [18] LOMANOVA, Y, 2021. Test Automation for Video Games: Things to Consider Before You Start - QATestLab Blog. QATestLab Blog | Blog About Software Testing [online]. [Accessed 10 December 2021]. Available from: <https://blog.qatestlab.com/2020/12/08/game-test-automation/>
- [19] GAMING, I., 2017. Most Critical Challenges in Mobile Game Testing. IXie Gaming [online]. [Accessed 10 December 2021]. Available from: <https://www.ixiegaming.com/blog/mobile-game-testing/>
- [20] RANA, K., 2021. Game Testing [online]. [Accessed 10 December 2021]. Available from: <https://artoftesting.com/game-testing>
- [21] HELPPI, V., 2020. Appium Tip #27: Using Appium for Mobile Game Testing [online]. [Accessed 10 December 2021]. Available from: <https://bitbar.com/blog/appium-tip-27-using-appium-for-mobile-game-testing/>
- [22] MARIO PLATFORMER [image]. In: *MarioWiki* [online]. [Accessed 10 December 2021]. Available from: <https://mario.fandom.com/>
- [23] About Unity Test Framework: Test framework: 1.1.30. *Test Framework* | 1.1.30 [online]. [Accessed 10 December 2021]. Available from: <https://docs.unity3d.com/Packages/com.unity.test-framework@1.1/manual/index.html>
- [24] Airtest project. *Airtest Project* [online]. [Accessed 10 December 2021]. Available from: <http://airtest.netease.com/>

- [25]Home. *GameDriver* [online]. [Accessed 10 December 2021]. Available from: <https://www.gamedriver.io/>
- [26]Altunity tools - unity test automation. *Altom* [online]. 9 December 2021. [Accessed 10 December 2021]. Available from: <https://altom.com/testing-tools/altunitytester/>
- [27] COLANTONIO, J., 2020. How to Automate Video Game Tests. Automation Testing Made Easy Tools Tips & Training [online]. [Accessed 10 December 2021]. Available from: <https://testguild.com/automate-video-games/>
- [28] NGUYEN, V., 2019. A Comparison of Automated Testing Tools [online]. [Accessed 10 December 2021]. Available from: <https://dzone.com/articles/a-comparison-of-automated-testing-tools>
- [29] TEAM, T. M., 2021. A Complete Automated Software Testing Tools Comparison [online]. [Accessed 10 December 2021]. Available from: <https://mentormate.com/blog/complete-guide-automated-software-testing-tools-comparison/>
- [30] TORBOCHKIN, B., 2013. Using Gestures in Mobile Game Design [online]. [Accessed 10 December 2021]. Available at: <https://www.gamedeveloper.com/design/using-gestures-in-mobile-game-design>
- [31]ODU, G.O., 2019. Weighting methods for multi-criteria decision making technique [online]. 11 September 2019. African Journals Online (AJOL). Available from: <http://dx.doi.org/10.4314/jasem.v23i8.7>.

Annexes

The objectives of this bachelor thesis were to create a prototype game and test scripts for each selected testing tool. Because of the game's size and complexity, it was decided to keep the source code on GitHub. Each of the three test scripts is attached as a separate file with the names shown below.

Annex A: Sample game's source code

As part of the thesis the mobile game was created using Udemy course. The game's source code is available at: <https://github.com/Lihodey13/Sampe-Game-Dengeon-Escape>.

Annex B: AltUnity Tester test script

Script covers defined test suites and is attached as a separate file with the name AltUnityTester_test.cs

Annex C: AirTestIDE test script

Script covers defined test suites and is attached as a separate file with the name AirTest_test.py

Annex D: GameDriver test script

Script covers defined test suites and is attached as a separate file with the name GameDriver_test.cs